Core/Src/main.c

```c
/* CPE 3160
 * John Oliver
 *
 * A4 - Clocks and Timers
 * Chris Amey and Alan Kusparmakov
 *
 * Analog Discovery Reading:
 * https://drive.google.com/file/d/1ridWuwsIxUjhIY48B9w1fNhTV8Lpmowz/view?usp=sharing
 */

#include "stdbool.h"
#include "stm32l476xx.h"
#include "stm32l4xx_hal.h"
#include "stm32l4xx_hal_gpio.h"
#include <stddef.h>
#include <stdint.h>

typedef enum { ALTERNATIVE = 2 } GpioMode;

typedef enum GpioState { HIGH = 1, LOW = 0 } GpioState;

void init()
{
  // gpio_init();
  // LCD_init(true, false, true, true, true, false);
}

// configure microcontroller clock output on PA8
void set_PA8_clock_MSI()
{
  // unselect all clocks
  // I think "selection" is just for blanket-setting an option to multiple clocks if you need to
  RCC->CFGR = ((RCC->CFGR & ~(RCC_CFGR_MCOSEL)));

  // select 0010 = MSI system clock
  RCC->CFGR |= RCC_CFGR_MCOSEL_0;

  // now configure selected clock(s)

  // enable GPIOA
  RCC->AHB2ENR |= RCC_AHB2ENR_GPIOAEN;

  // set mode
  GPIOA->MODER &= ~GPIO_MODER_MODE8_Msk;
  GPIOA->MODER |= (ALTERNATIVE << GPIO_MODER_MODE8_Pos);

  // allows STM to actively drive either low or high
  GPIOA->OTYPER &= ~(GPIO_OTYPER_OT8); // Push-pull output

  //
  GPIOA->PUPDR &= ~(GPIO_PUPDR_PUPD8); // no resistor
  GPIOA->OSPEEDR |= (GPIO_OSPEEDR_OSPEED8); // high speed

  // Set Pin 8's Alternate Function to be MCO
  GPIOA->AFR[1] &= ~(GPIO_AFRH_AFSEL8); // select MCO function
}

void TIM2_init()
{
  // APB1 = advanced peripheral bus
  // enable the CPU to TIM2 connection for configuring.
  // also connects a peripheral clock to TIM2
  RCC->APB1ENR1 |= (RCC_APB1ENR1_TIM2EN_Msk);

  // disable timer while configuring
  TIM2->CR1 &= ~(TIM_CR1_CEN_Msk);

  // CR1.DIR is up count by default
  // this is what we want

  TIM2->PSC = 0;
  TIM2->ARR = 799;

  // start at a known val
  TIM2->CNT = 0;

  // enable update interrupt
  // goes off whenever CNT == ARR
  TIM2->DIER |= TIM_DIER_UIE_Msk;

  // enable CCR1
  // goes off whenever CNT = CCR1
  TIM2->DIER |= TIM_DIER_CC1IE;

  // set the value to 25% of the duty cycle (800 / 4)
  TIM2->CCR1 = 200;

  // initialize the update interrupt to zero
  TIM2->SR &= ~(TIM_SR_UIF_Msk);

  // enable TIM2 in the NVIC
  // TIM2 is ISER[0] within NVIC.
```

```
  NVIC->ISER[0] = 1 << TIM2_IRQn;

  // can also do this
  // NVIC_EnableIRQ(TIM2_IRQn);

  // re-enable timer
  TIM2->CR1 |= TIM_CR1_CEN_Msk;

  // terminology:
  // DIER enable timer interrupt requests?
  // EGR generate timer events by software?
  // CR counter enable
  // UIE - update interrupt enable
  // CEN - counter enable
}

void set_PA0_wave_output()
{
  // enable GPIOA
  // here I am redundantly setting this flag so that there isn't a central
  // gpio init function that other functions rely on. either works for
  // proof of concept
  RCC->AHB2ENR |= RCC_AHB2ENR_GPIOAEN;

  // output mode
  GPIOA->MODER &= ~GPIO_MODER_MODE0_Msk;
  GPIOA->MODER |= 0b01 << GPIO_MODER_MODE0_Pos;

  // no push or pull resistors
  GPIOA->PUPDR &= ~GPIO_PUPDR_PUPD0_Msk;
  GPIOA->PUPDR |= 0b00 << GPIO_PUPDR_PUPD0_Pos;

  // output push-pull so that the pin drives 0 actively and 1 actively
  GPIOA->OTYPER &= ~GPIO_OTYPER_OT0_Msk;
  GPIOA->OTYPER |= 0b0 << GPIO_OTYPER_OT0_Pos;
}

void set_PA0(GpioState state)
{
  // GPIOA->ODR &= ~GPIO_ODR_OD0_Msk;
  // GPIOA->ODR |= state << GPIO_ODR_OD0_Pos;

  // the above can apparently be achieved also with a bit set-reset register
  // notice that we're setting the BSRR directly rather than with &= or |=
  // this is becasue BSRR is a write only register
  if (state == HIGH)
    GPIOA->BSRR = GPIO_BSRR_BS0;
  else
    GPIOA->BSRR = GPIO_BSRR_BR0;
}

int main()
{
  HAL_Init();
  set_PA8_clock_MSI(); // preserve existing clock output
  set_PA0_wave_output();
  TIM2_init();

  // blink the builtin LED to check that the program goes through
  {
    RCC->AHB2ENR |= RCC_AHB2ENR_GPIOAEN;
    GPIOA->MODER &= ~GPIO_MODER_MODE5;
    GPIOA->MODER |= GPIO_MODER_MODE5_0;

    while (true) {
      GPIOA->ODR ^= GPIO_ODR_OD5;
      HAL_Delay(1000);
    }
  }
}

void TIM2_IRQHandler()
{
  if (TIM2->SR & TIM_SR_UIF) {
    set_PA0(HIGH);

    // always clear flag
    TIM2->SR &= ~(TIM_SR_UIF);

    return;
  }

  if (TIM2->SR & TIM_SR_CC1IF) {
    set_PA0(LOW);

    // clear flag
    TIM2->SR &= ~(TIM_SR_CC1IF);
    return;
  }
}
```

Core/CMakeLists.txt

```
target_include_directories(${CMAKE_PROJECT_NAME} PRIVATE Inc)
add_subdirectory(Src)
```

Core/Inc/main.h

```c
/* USER CODE BEGIN Header */
/**
  ******************************************************************************
  * @file           : main.h
  * @brief          : Header for main.c file.
  *                   This file contains the common defines of the application.
  ******************************************************************************
  * @attention
  *
  * Copyright (c) 2026 STMicroelectronics.
  * All rights reserved.
  *
  * This software is licensed under terms that can be found in the LICENSE file
  * in the root directory of this software component.
  * If no LICENSE file comes with this software, it is provided AS-IS.
  *
  ******************************************************************************
  */
/* USER CODE END Header */

/* Define to prevent recursive inclusion -------------------------------------*/
#ifndef __MAIN_H
#define __MAIN_H

#ifdef __cplusplus
extern "C" {
#endif

/* Includes ------------------------------------------------------------------*/
#include "stm32l4xx_hal.h"
#include "stm32l4xx_nucleo.h"

/* Private includes ----------------------------------------------------------*/
/* USER CODE BEGIN Includes */

/* USER CODE END Includes */

/* Exported types ------------------------------------------------------------*/
/* USER CODE BEGIN ET */

/* USER CODE END ET */

/* Exported constants --------------------------------------------------------*/
/* USER CODE BEGIN EC */

/* USER CODE END EC */

/* Exported macro ------------------------------------------------------------*/
/* USER CODE BEGIN EM */

/* USER CODE END EM */

/* Exported functions prototypes ---------------------------------------------*/
void Error_Handler(void);

/* USER CODE BEGIN EFP */

/* USER CODE END EFP */

/* Private defines -----------------------------------------------------------*/
#define USART_TX_Pin GPIO_PIN_2
#define USART_TX_GPIO_Port GPIOA
#define USART_RX_Pin GPIO_PIN_3
#define USART_RX_GPIO_Port GPIOA
#define TMS_Pin GPIO_PIN_13
#define TMS_GPIO_Port GPIOA
#define TCK_Pin GPIO_PIN_14
#define TCK_GPIO_Port GPIOA
#define SWO_Pin GPIO_PIN_3
#define SWO_GPIO_Port GPIOB

/* USER CODE BEGIN Private defines */

/* USER CODE END Private defines */

#ifdef __cplusplus
}
#endif

#endif /* __MAIN_H */
```

Core/Src/CMakeLists.txt

```
target_sources(${CMAKE_PROJECT_NAME} PRIVATE main.c)
```